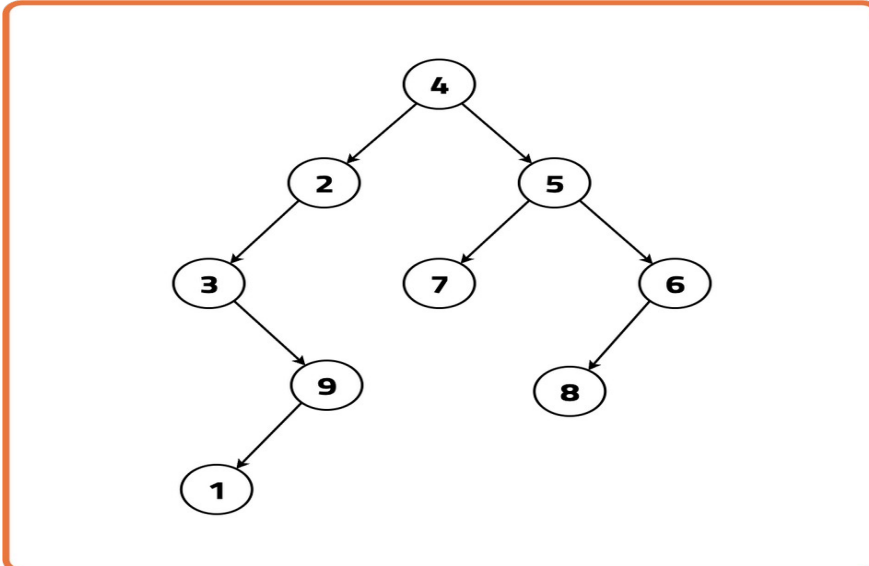


Preorder Traversal of Binary Tree

Problem Statement: Given the root of a Binary Tree, return the preorder, inorder and postorder traversal sequence of the given tree by making just one traversal.

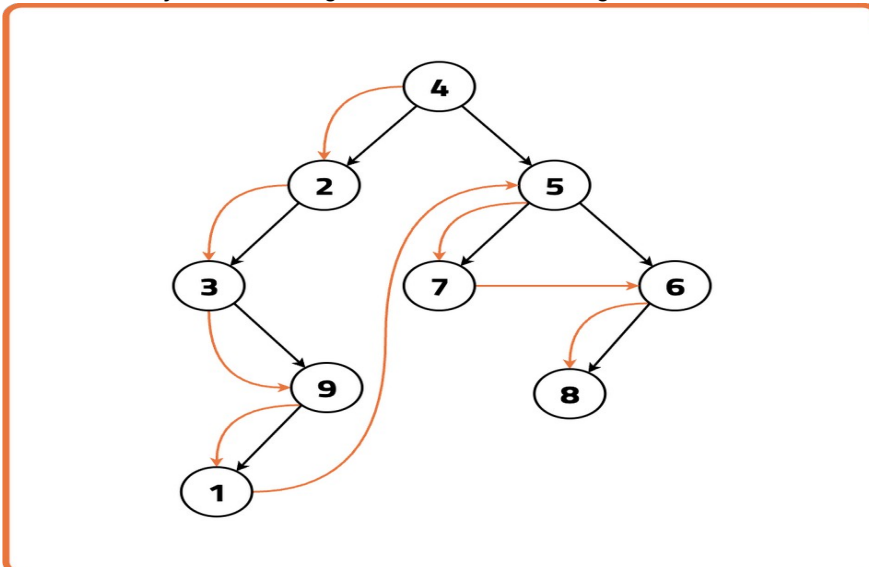
Examples

Input: Binary Tree: 4 2 5 3 -1 7 6 -1 9 -1 -1 8 -1 1

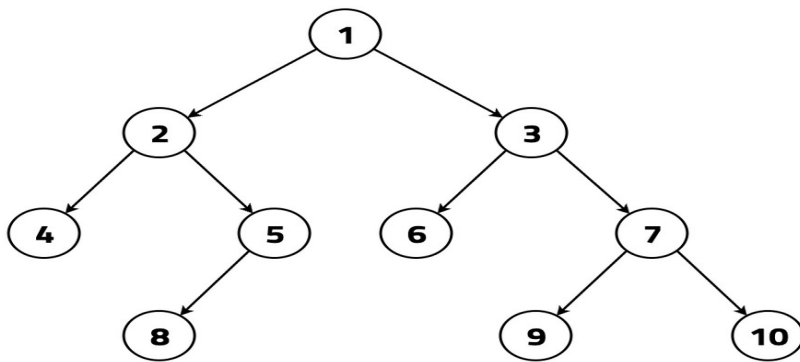


Output: Preorder: [4, 2, 3, 9, 1, 5, 7, 6, 8]

Explanation: We traverse the binary tree in the order of Root, Left then Right recursively resulting in the following traversal:

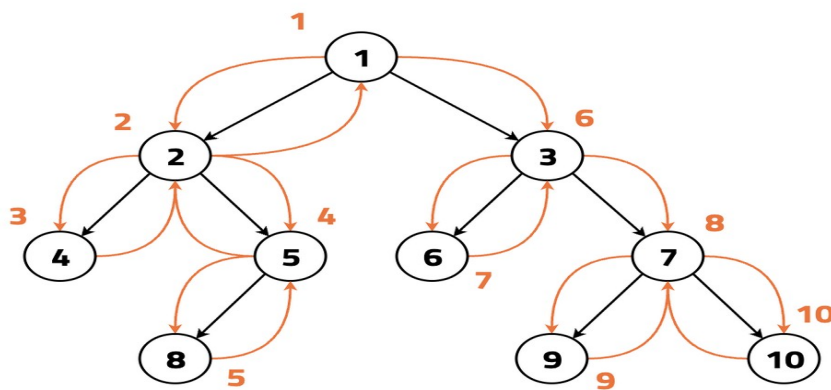


Input: Binary Tree: 1 2 3 4 5 6 7 -1 -1 8 -1 -1 -1 9 10



Output: Preorder: [1, 2, 4, 5, 8, 3, 6, 7, 9, 10]

Explanation: We traverse the binary tree in the order of Root, Left then Right recursively resulting in the following traversal:



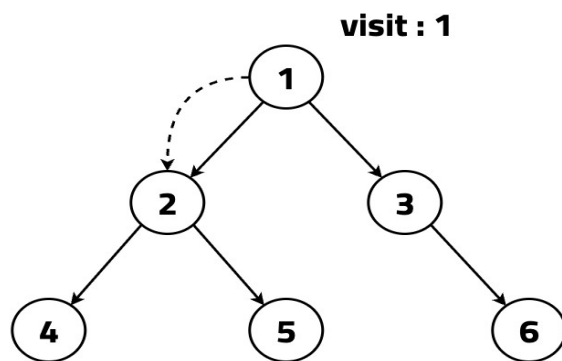
preorder arr = [1, 2, 4, 5, 8, 3, 6, 7, 9, 10]

Approach

Algorithm

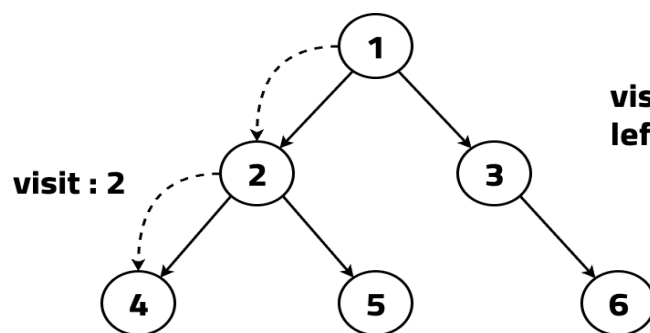
Preorder traversal is one of the depth-first traversal methods used to explore nodes in a binary tree. The algorithm first visits the root node then in the preorder traversal, we visit (ie. add to the array) the current node by accessing its value then we recursively traverse the left subtree in the same manner. We repeat these steps for the left subtree then when we return to the current node, we recursively travel to the right subtree in a preorder manner as well. The sequence of steps in preorder traversal follows: Root, Left, Right.

- Visit the current node and store its value (process the root first).
- Recursively traverse the left subtree by invoking the preorder function on the left child of the current node.
- After traversing the entire left subtree, we traverse the right subtree recursively. We once again invoke the preorder function, but this time on the right child of the current node.



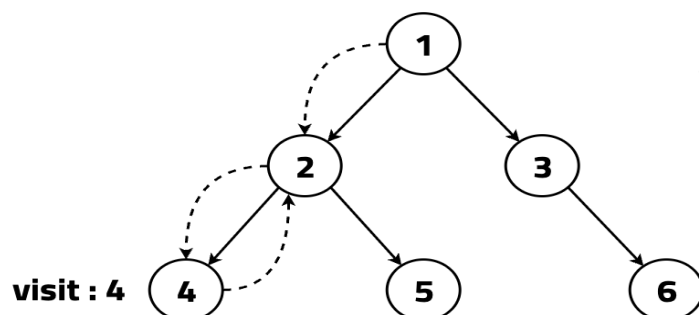
visit the current node
and call, reorder for
left subtree

preorder = [1]



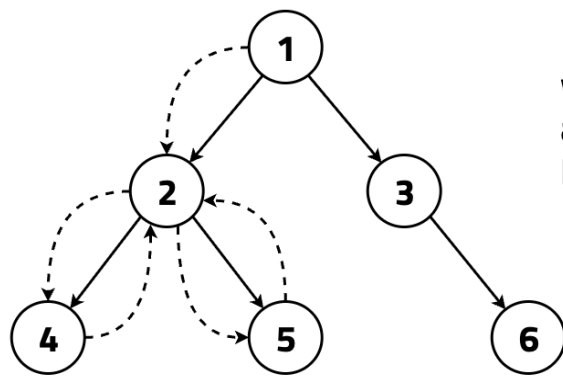
visit 2 and call preorder for
left subtree

preorder = [1, 2]



visit 4 and return as its
left and right children are
NULL

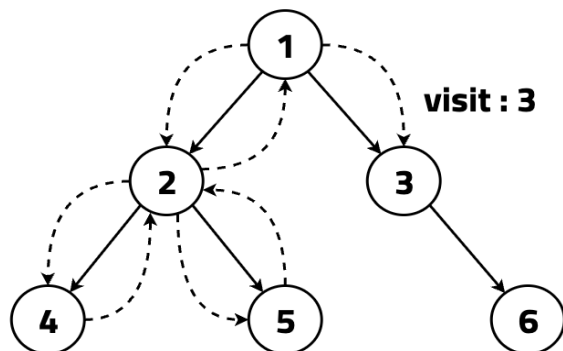
preorder = [1, 2, 4]



**visit 5 and return as its left
and right children are
NULL**

visit : 5

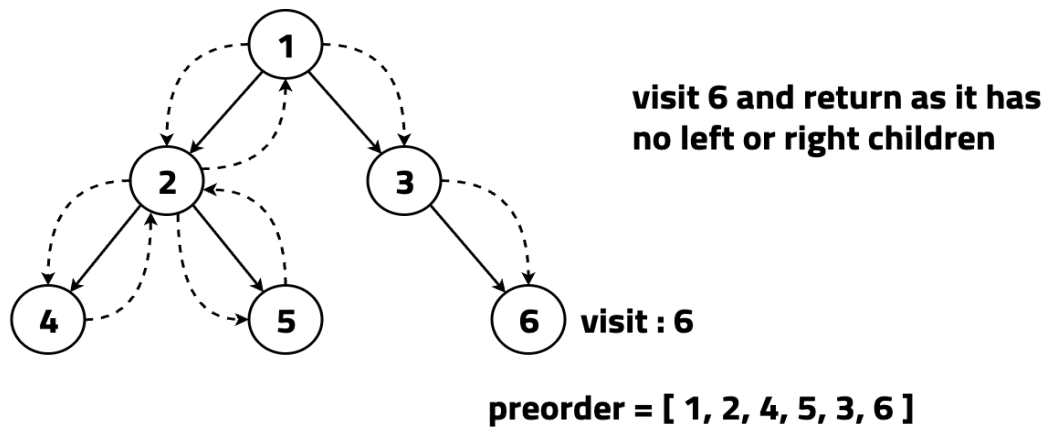
preorder = [1, 2, 4, 5]



visit : 3

**As we backtrack from 2
visit the right child of 1**

preorder = [1, 2, 4, 5, 3]



Code

CppJavaPythonJavascript

```
// Node structure for
// the binary tree
class Node {
    int data;
    Node left;
    Node right;

    // Constructor to initialize
    // the node with a value
    Node(int val) {
        data = val;
        left = null;
        right = null;
    }
}

// Solution class to perform preorder traversal
class Solution {

    // Function to perform preorder traversal
    // of the tree and store values in 'arr'
    public void preorder(Node root, java.util.List<Integer> arr) {
        // If the current node is NULL
        // (base case for recursion), return
        if (root == null) {
            return;
        }
        // Push the current node's
        // value into the vector
        arr.add(root.data);
        // Recursively traverse
        // the left subtree
        preorder(root.left, arr);
        // Recursively traverse
        // the right subtree
        preorder(root.right, arr);
    }
}
```

```

    }

    // Function to initiate preorder traversal
    // and return the resulting list
    public java.util.List<Integer> preOrder(Node root) {
        // Create an empty list to
        // store preorder traversal values
        java.util.List<Integer> arr = new java.util.ArrayList<>();
        // Call the preorder traversal function
        preorder(root, arr);
        // Return the resulting list
        // containing preorder traversal values
        return arr;
    }
}

// Main class
public class Main {
    public static void main(String[] args) {

        // Creating a sample binary tree
        Node root = new Node(1);
        root.left = new Node(2);
        root.right = new Node(3);
        root.left.left = new Node(4);
        root.left.right = new Node(5);

        // Getting preorder traversal
        Solution sol = new Solution();
        java.util.List<Integer> result = sol.preOrder(root);

        // Displaying the preorder traversal result
        System.out.print("Preorder Traversal: ");
        // Output each value in the
        // preorder traversal result
        for (int val : result) {
            System.out.print(val + " ");
        }
        System.out.println();
    }
}

```

Complexity Analysis.**Time Complexity: $O(N)$** , we process each node once for traversal.

Space Complexity: $O(N)$, extra space used for storing preorder traversal and recursive stack.